

Optimization with OpenTURNS

Problem Formulation, Algorithms and Visualization

June 12, 2026



OpenTURNS is widely used for uncertainty quantification, but it also provides a rich optimization framework.

Typical applications:

- Design optimization
- Parameter calibration
- Surrogate-assisted optimization
- Reliability-based design optimization
- Robust optimization

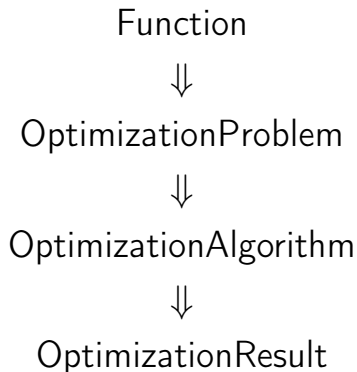
Optimization

Refer to **Optimization Algorithms**.

<code>OptimizationProblem(*args)</code>	Base class to define an optimization problem.
<code>NearestPointProblem(*args)</code>	Nearest point problem.
<code>LeastSquaresProblem(*args)</code>	Least-squares problem.
<code>experimental.LinearProblem(*args)</code>	Linear optimization problem.
<code>OptimizationAlgorithm(*args)</code>	Base class for optimization wrappers.
<code>OptimizationResult(*args)</code>	Optimization result.
<code>AbdoRackwitz(*args)</code>	Abdo-Rackwitz solver.
<code>Bonmin(*args)</code>	Bonmin MINLP solver.
<code>Ceres(*args)</code>	Interface to Ceres Solver.
<code>CMinpack(*args)</code>	Interface to CMinpack.
<code>Cobyla(*args)</code>	Constrained Optimization BY Linear Approximations solver.
<code>Dlib(*args)</code>	Base class for optimization solvers from the [dlib2009] library.
<code>experimental.EfficientGlobalOptimization(*args)</code>	Efficient Global Optimization algorithm.
<code>experimental.HiGHS(*args)</code>	Linear optimization solver.
<code>Ipopt(*args)</code>	Ipopt nonlinear optimization solver.
<code>MultiStart(*args)</code>	Multi-start optimization algorithm.
<code>NLOpt(*args)</code>	Interface to NLOpt.
<code>Pagmo(*args)</code>	Pagmo algorithms.
<code>SQP(*args)</code>	Sequential Quadratic Programming solver.
<code>TNC(*args)</code>	Truncated Newton Constrained solver.

Screenshot from OpenTURNS optimization documentation.

Optimization Architecture in OpenTURNS



Key idea:

- Same API for all optimizers
- Easy switching between algorithms
- Unified result object

Formulating an Optimization Problem

General constrained optimization problem:

$$\min_{\mathbf{x}} f(\mathbf{x})$$

subject to

$$g(\mathbf{x}) \leq 0$$

$$h(\mathbf{x}) = 0$$

$$\mathbf{l} \leq \mathbf{x} \leq \mathbf{u}$$

Mathematics	OpenTURNS
Objective function $f(\cdot)$	Function
Constraints $g(\cdot), h(\cdot)$	Functions
Optimization variables $\mathbf{x}$	Point
Bounds $[\mathbf{l}, \mathbf{u}]$	Interval
Optimization problem	OptimizationProblem

Possibility to solve **multi-objective** optimization problem.

Example: Rosenbrock Problem

Objective function:

$$f(x_1, x_2) = (1 - x_1)^2 + 100(x_2 - x_1^2)^2$$

```
import openturns as ot

f = ot.SymbolicFunction(
    ["x1", "x2"],
    ["(1-x1)^2 + 100*(x2-x1^2)^2"]
)
```

Possibility to wrap any complex optimization function with PythonFunction:

```
def f_(x):
    x1 = x[0]
    x2 = x[1]
    y = (1-x1)**2+100*(x2-x1**2)**2
    return [y]

f = ot.PythonFunction(2,1,f_)
```

Creating the Optimization Problem

```
problem = ot.OptimizationProblem(f)

problem.setMinimization(True)

problem.setBounds(
    ot.Interval(
        [-2.0, -2.0],
        [ 2.0,  2.0]
    )
)
```

Possible ingredients for an OptimizationProblem:

- objective function (or multi-objective function in a list form)
- inequality constraints
- equality constraints
- bounds

Optimization algorithms available in OpenTURNS

Algorithms	Package	Functionalities
SQP	oT	Gradient-based
Cobyla	oT	Gradient-based
TNC	oT	Gradient-based
AbdoRackwitz	oT	Level-Function
IpOpt	IpOpt	Gradient-based
CMinpack	CMinpack	Least-squares
NLOpt (42)	NLOpt	Local/global (w. or w.o. gradient)
BonMin (6)	BonMin	Mixed integer (MINLP)
Pagmo (19)	Pagmo	Evolutionary / multi-objective / MINLP
Ceres (6)	Ceres	Least-squares
Dlib (8)	Dlib	Gradient-based/Least-squares
MultiStart	oT	Enable multi-start of previous algorithms

Total: **87 optimization algorithms**

- Unified interface
- Easy benchmarking
- Same result object

How to get algorithms ?

Example from Pagmo:

Pagmo provides the following global **heuristics**:

Algorithm	Description	Multi-objective	MINLP	Batch
gaco	Extended Ant Colony Optimization	no	yes	yes
de	Differential Evolution	no	no	no
sade	Self-adaptive DE (jDE and iDE)	no	no	no
de1220	Self-adaptive DE (de_1220 aka pDE)	no	no	no
gwo	Grey wolf optimizer	no	no	no
ihs	Improved Harmony Search	no	yes	no
pso	Particle Swarm Optimization	no	no	no
pso_gen	Particle Swarm Optimization Generational	no	no	yes
sea	(N+1)-ES Simple Evolutionary Algorithm	no	no	no
sga	Simple Genetic Algorithm	no	yes	no
simulated_annealing	Corana's Simulated Annealing	no	no	no
bee_colony	Artificial Bee Colony	no	no	no
cmaes	Covariance Matrix Adaptation Evo. Strategy	no	no	yes
xnes	Exponential Evolution Strategies	no	no	no
nsga2	Non-dominated Sorting GA	yes	yes	yes
moea	Multi-objective EA with Decomposition	yes	no	no
moea_gen	Multi-objective EA with Decomposition Gen.	yes	no	yes
mhaco	Multi-objective Hypervolume-based ACO	yes	yes	yes
nspso	Non-dominated Sorting PSO	yes	no	yes

Only gaco and ihs natively support constraints, but for the other algorithms constraints are emulated through penalization. For mhaco, the initial population must satisfy constraints, else it is built by bootstrap on valid points with the same population size as the one provided. Some algorithms support batch evaluation, see `setBlockSize()`. Default parameters are available in the [ResourceMap](#) for each algorithm, refer to the correspondings keys in the Pagmo [documentation](#).

Running an optimization algorithm

Example with COBYLA:

```
algo = ot.Cobyala(problem)

algo.setStartingPoint([0.0,0.0])

algo.run()

result = algo.getResult()
```

The algorithm controls:

- Stopping criteria:
 - Maximum number of evaluations
 - Absolute and relative tolerances
 - etc.
- Starting point

Global optimization with multiStart

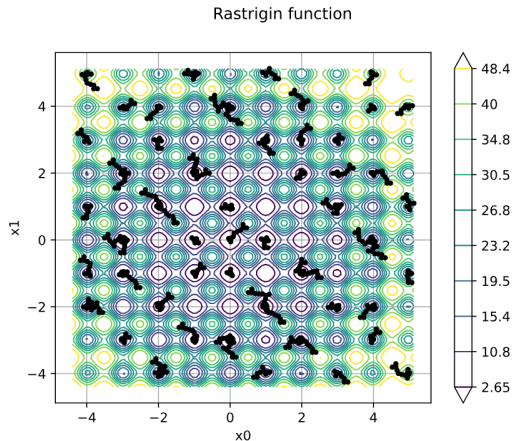
Local optimizers may converge to local minima.

MultiStart launches several local searches: $\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \dots, \mathbf{x}^{(N)}$

```
algo = ot.MultiStart(  
    ot.Cobyla(problem),  
    startingSample  
)
```

Benefits:

- Adapted to multimodal problems
- More robust convergence
- Parallel exploration



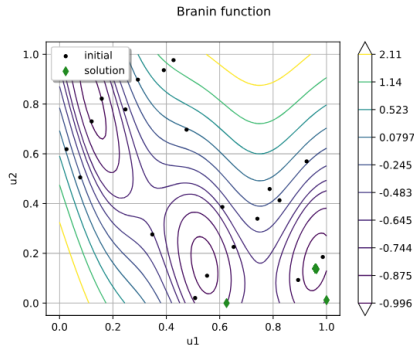
Surrogate-assisted optimization

In case of computationally costly function: possibility to use Efficient Global Optimization that uses Gaussian Process Regression

```
algo = otxp.  
    EfficientGlobalOptimization(  
        problem, gpr.getResult())  
algo.setMaximumCallsNumber(30)  
algo.run()  
result = algo.getResult()
```

Benefits:

- Global optimization
- Low number of function evaluations



Visualization of optimization Results

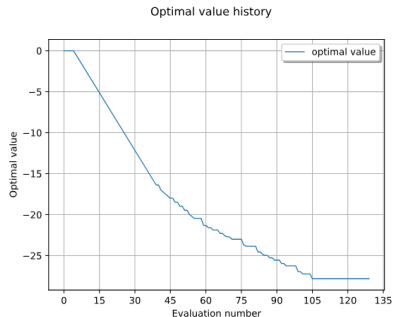
Optimization history is stored automatically.

```
result.getInputHistory()
```

```
result.getOutputHistory()
```

Useful visualizations:

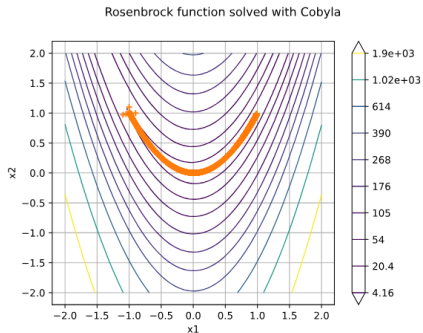
- Objective value vs evaluation number
- Constraint violation vs evaluation number
- Evolution of design variables
- Convergence diagnostics



Visualization of the Optimization Path

For two-dimensional problems we can visualize:

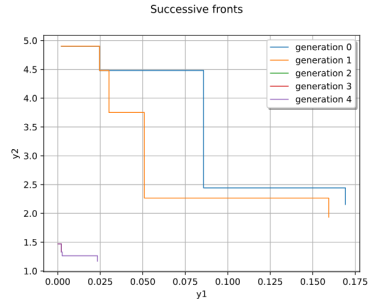
- Objective contours
- Feasible domain
- Optimization trajectory
- Final optimum



Visualization of the Optimization Results

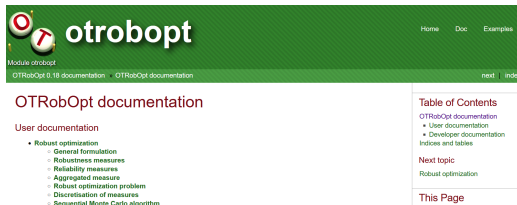
Dedicated features for multi-objective problems:

- Pareto Front
- Successive Pareto fronts along the iterations
- Populations



OTRobOpt: dedicated OpenTURNS module for robust optimization:

- Optimize under uncertainty rather than nominal conditions,
- Evaluate customizable robustness and risk measures,
- End-to-end workflow: problem definition → robustness evaluation → optimization



OpenTURNS enables seamless coupling between optimization and UQ.

- Search for design point in reliability (FORM / SORM)
- Hyperparameters tuning of Machine Learning algorithms (GPR, PCE)
- Calibration of model parameters (least-square problem)
- Fitting problems (maximum likelihood)
- etc.

Take-home messages:

- Unified optimization API
- Multiple optimization backends
- Rich visualization capabilities
- Natural integration with UQ workflows

Do you want to try ?

→ go to quick start example

▶ link

Quick start guide to optimization

In this example, we perform the optimization of the Rosenbrock test function.

Let $a, b \in \mathbb{R}$ be parameters. The Rosenbrock function is defined by

$$f(x_1, x_2) = (a - x_1)^2 + b(x_2 - x_1^2)^2$$

for any $x \in \mathbb{R}^2$. This function is often used with $a = 1$ and $b = 100$. In this case, the function has a single global minimum at:

$$x^* = (1, 1)^T.$$

This function has a nonlinear least squares structure.

References

- Rosenbrock, H.H. (1960). "An automatic method for finding the greatest or least value of a function". The Computer Journal. 3 (3): 175–184.

Definition of the function

```
import openturns as ot
import openturns.viewer as otv
```

```
rosenbrock = ot.SymbolicFunction(["x1", "x2"], ["(1-x1)^2+100*(x2-x1^2)^2"])
```

```
x0 = [-1.0, 1.0]
```

```
xexact = ot.Point([1.0, 1.0])
```

```
lowerbound = [-2.0, -2.0]
upperbound = [2.0, 2.0]
```

Questions ?